

Problema Flight Discount — CSES

Resolução com o Algoritmo de Dijkstra

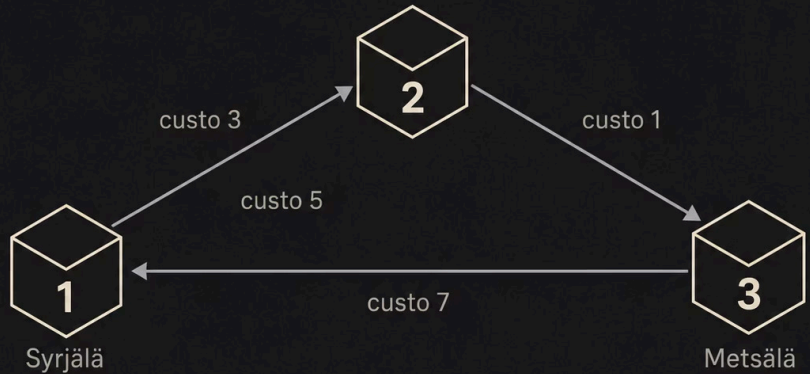
RESOLUÇÃO PROBLEMAS COM GRAFOS

José Holanda e João Victor Sampaio

Descrição do Problema

Exemplo do Problema

Encontre o menor custo para ir da cidade 1 (Syrjälä) até a cidade n (Metsälä) usando um cupom de desconto em exatamente um voo.



Possíveis rotas e custos:

- 1 → 2 → 3 (sem cupom): $3 + 1 = 4$

Cupom em 1 → 2: $\lfloor 3/2 \rfloor + 1 = 1 + 1 = 2$

- 1 → 3 (sem cupom): 7

Cupom em 1 → 3: $\lfloor 7/2 \rfloor = 3$

- 1 → 2 → 1 → 3 (com cupom): $3 + \lfloor 5/2 \rfloor + 7 = 12$ (não é ótimo)



Melhor rota:

1 → 2 → 3
usando o cupom
no voo 1 → 2

Custo mínimo:

2

Cenário

- **n** cidades conectadas por voos **unidirecionais**
- Cada voo possui um **custo positivo**
- Objetivo: menor custo da cidade **1** até a cidade **n**

Cupom de Desconto

- Disponível **exatamente uma vez** em qualquer voo do percurso
- Custo x torna-se $\lfloor x/2 \rfloor$
- Objetivo: **minimizar o custo total** da viagem

📌 Exemplo: voo de custo 5 → com cupom: custo 2

Modelagem como Grafo

Vértices

Cada cidade é um vértice $v \in \{1, \dots, n\}$

Arestas Direcionadas

Cada voo é uma aresta (u, v) com direção fixa

Pesos

Custo do voo = peso da aresta, com valores até 10^9

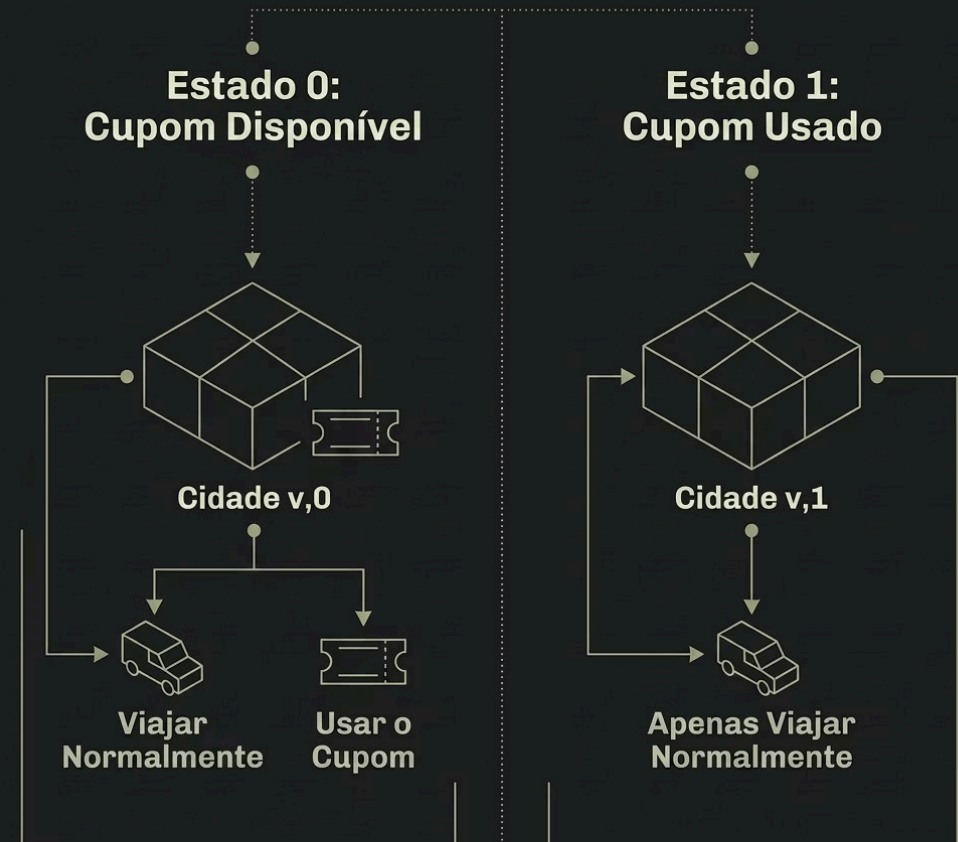
O problema se torna: encontrar o **caminho mínimo** de 1 a n, podendo aplicar o desconto em **exatamente uma aresta** do caminho.

O Desafio Principal

Por que Dijkstra tradicional não basta?

No Dijkstra clássico, cada vértice armazena um único valor de distância mínima. Aqui, chegar à cidade v com o cupom **disponível** é fundamentalmente diferente de chegar com o cupom **já usado**.

⚠ Dois estados distintos com o mesmo custo podem levar a resultados finais completamente diferentes.



Estratégia: Dois Estados por Cidade

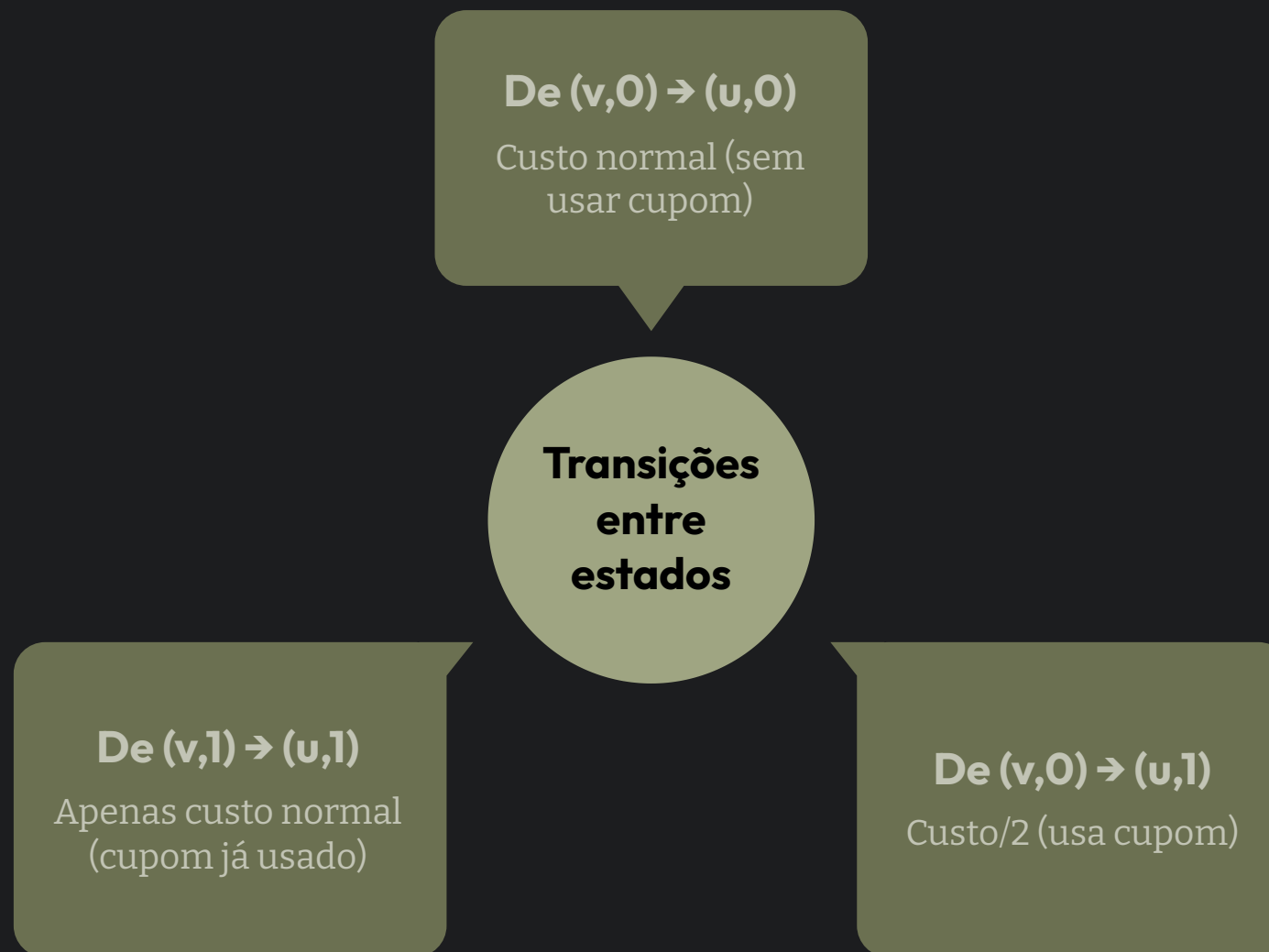
Representamos cada cidade v em dois estados distintos, capturando se o cupom já foi consumido ou não.

Estado 0 — Cupom Disponível

$\text{dist}[v][0]$ = menor custo para chegar em v **sem** ter usado o cupom

Estado 1 — Cupom Utilizado

$\text{dist}[v][1]$ = menor custo para chegar em v **após** usar o cupom em algum voo anterior



O diagrama acima ilustra as três transições possíveis entre os estados de cada cidade durante a execução do algoritmo.

Aplicação do Dijkstra Adaptado

Transições de Estado

Ao relaxar a aresta (v, u) de custo c :

Viagem normal

$\text{dist}[v][0] \rightarrow \text{dist}[u][0] + c$

Usar o cupom

$\text{dist}[v][0] \rightarrow \text{dist}[u][1] + \lfloor c/2 \rfloor$

Cupom já usado

$\text{dist}[v][1] \rightarrow \text{dist}[u][1] + c$

Pseudocódigo Simplificado

inicializar $\text{dist}[i][0] = \text{dist}[i][1] = \infty$ para todas as cidades i

$\text{dist}[1][0] = 0$

$\text{fila} = \{(0, 1, 0)\}$ // (custo, cidade, estado)

enquanto fila não vazia:

 extrair (d, u, s) com menor d

 se $d > \text{dist}[u][s]$: continuar

 para cada vizinho v de u com custo c :

 // Viajar normalmente (mantém o estado s)

 se $d + c < \text{dist}[v][s]$:

$\text{dist}[v][s] = d + c$

 inserir $(\text{dist}[v][s], v, s)$ na fila

 // Usar cupom (só é possível se $s == 0$)

 se $s == 0$ e $d + \lfloor c/2 \rfloor < \text{dist}[v][1]$:

$\text{dist}[v][1] = d + \lfloor c/2 \rfloor$

 inserir $(\text{dist}[v][1], v, 1)$ na fila

resposta = $\min(\text{dist}[n][0], \text{dist}[n][1])$

Complexidade do Algoritmo

$2n$

Estados totais

Cada cidade possui 2 estados: cupom disponível e cupom usado

$3m$

Arestas efetivas

Cada voo gera até três transições de estado: manter o cupom ($0 \rightarrow 0$), usar o cupom ($0 \rightarrow 1$) ou viajar pós-uso ($1 \rightarrow 1$)

$O(m \log n)$

Complexidade temporal

Devido ao uso da *PriorityQueue* nativa do Java (Lazy Deletion), a fila pode armazenar até $O(m)$ caminhos simultaneamente.

Logo, o custo total é $O(m \log m)$.

$O(n+m)$

Complexidade espacial

Armazenamento do grafo e das distâncias nos dois estados

O fator constante 2 é absorvido pela notação assintótica, mantendo a solução dentro dos limites do problema.

Casos Especiais e Robustez



Cupom no primeiro ou último voo

O algoritmo considera **todas as posições** possíveis para o cupom, utilizando fila de prioridade para garantir sempre o menor custo



Melhor caminho muda com desconto

O caminho ótimo **sem cupom** pode diferir do caminho ótimo **com cupom** — ambos são explorados



Ciclos e pesos grandes

Pesos positivos garantem que Dijkstra funcione; custos até **10^9** cabem em `long long`



Escala do problema

Até **10^5 vértices** e **2×10^5 arestas** — a complexidade $O((n+m)\log n)$ é suficiente

Conclusão

Pontos-Chave

- Problema modelado como grafo ponderado direcionado
- Dois estados por cidade capturam o uso do cupom
- Dijkstra adaptado garante solução ótima
- Complexidade $O(m \log n)$ – eficiente para os limites
- Técnica generalizável para caminhos mínimos com estados

Perguntas Frequentes

Por que não usar Bellman-Ford?

Mais lento: $O(n \cdot m)$. Dijkstra é preferível com pesos não-negativos.

E se houver múltiplos cupons?

Generaliza para k estados: $\text{dist}[v][0..k]$. Complexidade $O(k \cdot (n+m) \log(kn))$.

Por que $\text{dist}[n][1]$ é a resposta?

O cupom deve ser usado exatamente uma vez; $\text{dist}[n][1]$ cobre todos os caminhos válidos.